Turnstile State Machine

1 . Carrier type

Define the two gate positions as a free type. Using a named type (rather than a boolean) makes the state schema self-documenting and prevents silent confusion between the two values.

$$Gate ::= locked \mid unlocked$$

2 . State schema

The state schema declares every persistent variable and states the invariant that every reachable state must satisfy. Here the invariant records that the passage count is always non-negative — a property that every operation must preserve.

TurnstileState

$$gate : Gate$$
$$passed : \mathbb{N}$$

$$passed \geq 0$$

3 . Init schema

The Init schema pins down the initial configuration. Writing theta TurnstileState' = ... packages the entire after-state as a single binding, which is the Spivey convention for initial-state predicates. It says: the initial state is exactly the turnstile in the locked position with a zero passage count.

TurnstileInit

$$TurnstileState'$$

$$gate' = locked$$
$$passed' = 0$$

4 a . Delta operation : Coin

Inserting a coin unlocks the gate. The Delta inclusion brings both the before-state (gate, passed) and the after-state (gate', passed') into scope. The precondition requires the gate to be locked before the coin is accepted; the postcondition sets gate' to unlocked and leaves the passage count unchanged.

Coin

$\Delta TurnstileState$

$gate = locked$
$gate' = unlocked$
$passed' = passed$

4 b . Delta operation : Push

Pushing the bar when the gate is unlocked lets someone through. The gate relocks and the passage count increments by one. The input decoration is not needed here because no external value enters the system — the push is a pure physical event. An output result! reports the new passage count so the caller can log it without a separate query.

Push

$\Delta TurnstileState$
$result! : \mathbb{N}$

$gate = unlocked$
$gate' = locked$
$passed' = passed + 1$
$result! = passed'$

5 . Xi query : IsLocked

A query operation uses Xi to assert that no state variable changes. The input is not required; the output status! carries the answer. Because Xi TurnstileState guarantees *gate* = *gate*' and *passed* = *passed*', the caller learns the current state without any risk of mutation.

IsLocked

$\Xi TurnstileState$
$status! : Gate$

$status! = gate$